Technical Report: Rice Classification on STM32F746G

Project Phase: Part-2
Target Platform: STM32F746G Discovery

1. Introduction

1.1 Project Overview

- Concept: Development of an end-to-end Machine Learning pipeline for classifying rice varieties on a resource-constrained microcontroller.
- Core Technology: Utilization of Tiny EfficientNet, a specialized version of the EfficientNet architecture optimized for edge computing.
- Target: Real-time identification of 5 rice types: Arborio, Basmati, Ipsala, Jasmine, and Karacadag.

1.2 Objectives

- Miniaturization: Scaling down a state-of-the-art CNN (EfficientNet) to fit within the memory limits of the STM32F7 series.
- Quantization: Implementing INT8 Post-Training Quantization to ensure efficient execution on hardware without a dedicated NPU.
- System Validation: Achieving a balance between inference speed and classification accuracy on-device.

1.3 System Constraints and Target Platform

- Hardware: STM32F746G-DISCO board featuring an ARM Cortex-M7 core.
- Memory Footprint: * RAM: Limited available SRAM for the Tensor Arena (set to 200 KB in the project).
    - Flash: Storage of the model weights as a C++ byte array.
- Software Stack: TensorFlow Lite for Microcontrollers (TFLM) and CMSIS-RTOS.

2. Problem Definition

2.1 Problem Statement

- Need: Manual inspection of rice grains for variety and quality is labor-intensive, subjective, and prone to human error.

- Solution: An automated, high-speed classification system utilizing Edge-AI to provide instant, objective analysis.
- Challenge: Implementing a complex deep learning model (EfficientNet) on a system with severely restricted RAM and Flash memory without a significant loss in classification accuracy.

2.2 Input and Output Requirements

- Input Pipeline:
  - Data Type: Grayscale image data.
  - Resolution: 96x96 pixels (Single channel).
  - Data Size: 9,216 bytes per image frame.
- Output Specifications:
  - Type: Integer class ID (0 to 4).
  - Labels: {0: Arborio, 1: Basmati, 2: Ipsala, 3: Jasmine, 4: Karacadag}.
  - Feedback: Inference results transmitted back to the host PC via UART.

2.3 Performance and Memory Constraints

- Static Memory (Flash): The compiled model must fit within the internal Flash of the STM32F746G (1 MB).
- Dynamic Memory (RAM): * Tensor Arena: Hard cap of 200 KB
  - This arena must house all intermediate activations, input/output tensors, and temporary buffers for convolution operations.
- Latency: Inference must be fast enough to handle serial data transmission without significant bottlenecks, targeting efficient execution on the 216 MHz Cortex-M7 processor.

3. Dataset Description

3.1 Dataset Source

- Original Source: Rice Image Dataset provided by Murat Koklu.
- Primary Citation: * Koklu, M., Cinar, I., & Taspinar, Y. S. (2021). Classification of rice varieties with deep learning methods. Computers and Electronics in Agriculture, 187, 106285. https://doi.org/10.1016/j.compag.2021.106285
- Availability: Publicly hosted at https://www.muratkoklu.com/datasets/.

3.2 Data Format and Features

- Scope: The full dataset contains 75,000 images (15,000 per variety).
- Varieties: Arborio, Basmati, Ipsala, Jasmine, and Karacadag.
- Embedded Optimization: For this project, images were downsampled to 96x96 pixels and converted to Grayscale to accommodate the STM32F746G's memory constraints.

- Feature Focus: The model relies on morphological (shape) and textural features inherent in the grain structures to differentiate between types.

3.3 Preprocessing and Augmentation

- Normalization: Input pixel values scaled to a [0, 1] range (and subsequently quantized to INT8 for deployment).
- Augmentation Strategy: * Random Rotation: 0.05 factor to ensure rotation invariance.
    - Random Zoom: 0.1 factor to simulate varying camera-to-grain distances.
- Calibration: A representative subset of the dataset was used during TFLite conversion to define the dynamic range of activations for the integer-only pipeline.

3.4 Train / Validation / Test Split

- Training Set: 17,284 images (80%) used for model weight optimization.
- Validation Set: 4,323 images (20%) used for epoch-wise performance monitoring and preventing overfitting.
- On-Device Test Set: A random subset of 1,000 images was utilized via the test.py serial transmission tool to verify real-world inference accuracy on the STM32 hardware.

4. Model Development on PC

4.1 Model Architecture

- Core Design: A custom Tiny EfficientNet utilizing Mobile Inverted Bottleneck Convolution (MBConv) blocks.
- Feature Enhancement: Integration of Squeeze-and-Excitation (SE) blocks to adaptively recalibrate channel-wise feature responses.
- Layer Breakdown:
    - Stem: 3x3 Convolution with 16 filters and stride 2.
    - MBConv Blocks: Five blocks with increasing filter depths (16, 24, 24, 40, 40) and varied strides (1 or 2).
    - Head: 1x1 Convolution with 64 filters, followed by Global Average Pooling and a Softmax output layer for 5 classes.
- Input Shape: 96x96x1 (Grayscale).

```python
def build_tiny_efficientnet(input_shape=(96,96,1), classes=5):
    inputs = tf.keras.layers.Input(shape=input_shape)

    # Stem
    x = tf.keras.layers.Conv2D(16, 3, strides=2, padding="same", use_bias=False)(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    # EfficientNet blocks (MBConv)
    x = mbconv(x, 16, 1)
    x = mbconv(x, 24, 2)
    x = mbconv(x, 24, 1)
    x = mbconv(x, 40, 2)
    x = mbconv(x, 40, 1)

    # Head
    x = tf.keras.layers.Conv2D(64, 1, use_bias=False)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    outputs = tf.keras.layers.Dense(classes, activation="softmax")(x)

    return tf.keras.Model(inputs, outputs)
```

4.2 Training Environment (Python, Frameworks)

- Language: Python 3.
- Frameworks: TensorFlow and Keras for model construction and training.
- Data Handling: tf.keras.preprocessing for directory-based image loading and tf.data.AUTOTUNE for optimized pipeline prefetching.

4.3 Training Procedure

- Data Preparation:
    - Images resized to 96x96 pixels.
    - Pixel normalization to the range [0, 1] via float32 casting.
- Optimization: Adam optimizer paired with Categorical Cross-entropy loss.
- Batch Configuration: Batch size of 32.
- Regularization: Use of Batch Normalization layers and data augmentation (rotation and zoom) to ensure model stability and generalization.

4.4 Baseline Performance (Float Model)

- Benchmark: High-precision floating-point validation accuracy serves as the reference point for evaluating subsequent quantization loss.
- Objective: Ensure the float model achieves near-perfect classification on the PC environment before transitioning to the restricted embedded environment.

5. Model Quantization

5.1 Quantization Strategy

The transition from a high-performance PC environment to a resource-constrained microcontroller required a rigorous Post-Training Quantization (PTQ) process. We converted the model's 32-bit floating-point weights and activations into 8-bit integers (INT8). This wasn't merely a compression technique but a Full Integer Enforcement strategy. By mapping all mathematical operations to integer arithmetic, we bypassed the need for slow floating-point emulation, directly leveraging the hardware efficiency of the STM32's Cortex-M7 core.

```
# Representative dataset generator for INT8 Quantization
def representative_data_gen():
    for i in range(rep_images.shape[0]):
        yield [rep_images[i:i+1]]  # batch of 1, 4D tensor

# TFLite Conversion Configuration
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.uint8   # input quantized
converter.inference_output_type = tf.uint8  # output quantized

tflite_model = converter.convert()
```

5.2 TFLite Model Generation and Calibration

The success of the quantization hinged on the Calibration Set. We discovered that the volume of data used to determine the dynamic range of activations was critical. Our Refinement Iteration showed that while an initial attempt with 20 images per class led to significant errors and model failure, increasing the calibration set to 1,000 images per class stabilized the model. We utilized the tf.lite.Optimize.DEFAULT flag during generation to strike an optimal balance between minimizing Flash memory usage and maintaining the structural integrity of the EfficientNet layers.

5.3 Performance Evaluation and Memory Gains

Before hardware deployment, we performed a Validation step using a PC-based TFLite Interpreter. This confirmed that while some precision was lost, distinct classes like "Ipsala" and "Karacadag" maintained near 100% accuracy. The Size Reduction was substantial, achieving a ~4x decrease in total model size. This optimization was the primary driver in ensuring the model could fit within the 1MB Flash limit and operate within the 200 KB Tensor Arena—the peak RAM allocated in the Rice.cpp source code. Ultimately, the quantized system reached an on-board accuracy of 67.50%.

6. Model Conversion for Embedded Deployment

6.1 TFLite to C++ Conversion

Because microcontrollers cannot "load" a .tflite file from a standard file system like a PC, the model had to be embedded directly into the firmware. We used Automated Scripting via a Python tool to perform a Binary-to-Hex Transformation. This converted the binary model file into a massive hexadecimal array, effectively turning the neural network into a C++ source file that the compiler could integrate into the final executable.

6.2 Optimized Storage

To protect the system's limited RAM, we utilized the const qualifier when declaring the model array (e.g., const unsigned char g_efficientnet_model_data[]). This is a vital architectural decision: it flags the data for storage in Internal Flash Memory (ROM) instead of RAM. By doing so, we preserved the 216 KB of SRAM exclusively for the Tensor Arena and system operations. The TensorFlow Lite Micro interpreter is specifically designed for this, utilizing Direct Addressing to read weights straight from the Flash without needing to copy them into RAM first.

6.3 Project Integration and Safety Guards

The output of the conversion consists of two files: model_data.h, which provides the extern declaration for system-wide access, and model_data.cc, which houses the hex values of the network architecture and weights. In the app_main function of Rice.cpp, we perform Pointer Mapping to link the TFLM framework to the Flash-resident model. Finally, a Compatibility Guard performs a runtime check against the TFLITE_SCHEMA_VERSION. This ensures the model was compiled with a library version compatible with the firmware, preventing potential crashes during initialization.

7. Deployment on STM32F746G

7.1 Hardware Overview

The system is deployed on the STM32F746G-Discovery board, which serves as a high-performance entry point for embedded machine learning. At its core is the ARM Cortex-M7 processor running at 216 MHz, featuring a Level 1 cache and a Floating Point Unit (FPU). While the hardware supports floating-point operations, the project utilizes the DSP capabilities of the M7 core to execute INT8 quantized operations, which provides a significant boost in inference speed and a reduction in power consumption compared to standard float32 implementations.

7.2 Software Environment and Libraries

The firmware architecture is built upon the CMSIS-OS2 (FreeRTOS) middleware, providing a multi-threaded environment where the AI inference task is isolated from system management. The TensorFlow Lite for Microcontrollers (TFLM) library is integrated into the C++ environment to handle the model's execution. Communication with the host PC is established via a high-speed UART interface at 115200 baud, allowing the board to receive raw pixel data and transmit classification results in real-time.

7.3 TensorFlow Lite Micro Setup

To optimize the Flash memory footprint, a MicroMutableOpResolver<16> was utilized rather than loading the entire TFLite library. This "surgical" approach involves manually registering only the specific kernels required by the Tiny EfficientNet architecture, such as Conv2D, DepthwiseConv2D, Logistic (for the Sigmoid activation in SE blocks), and StridedSlice. This selective registration ensures that the final binary remains small enough to reside comfortably in the internal Flash alongside the model weights.

```
int app_main(void)
{
    // Optimized resolver to save Flash memory
    static tflite::MicroMutableOpResolver<16> resolver;

    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddAdd();
    resolver.AddMul();
    resolver.AddMean();
    resolver.AddFullyConnected();
    resolver.AddSoftmax();
    resolver.AddReshape();
    resolver.AddLogistic(); // Required for SE Blocks
    resolver.AddPack();
    resolver.AddStridedSlice();
    resolver.AddShape();
    resolver.AddQuantize();

    const tflite::Model* model = tflite::GetModel(g_efficientnet_model_data);
```

7.4 Tensor Allocation and Memory Management

The most critical deployment challenge was managing the Tensor Arena, which was statically allocated as a 200 KB uint8_t array. During the AllocateTensors() phase, the TFLM interpreter maps this memory block to accommodate the lifetime of every intermediate activation and temporary buffer. Because EfficientNet's inverted bottleneck blocks are memory-intensive, the model's width and head filters were iteratively scaled down to ensure that the peak memory pressure never exceeded the 204,800-byte threshold, preventing runtime memory corruption.

7.5 Application Inference Loop

The application logic follows a strict "ping-pong" synchronization with the host. Upon successful initialization, the board transmits a [READY] signal over serial. It then blocks until it receives exactly 9,216 bytes of grayscale data, which are written directly into the input tensor's memory address. The interpreter->Invoke() command triggers the inference process, after which the firmware performs a Softmax comparison on the output tensor. The final classification index (0-4) is then printed back to the console as a RESULT:X string, completing the edge-AI cycle.

```
static __NO_RETURN void inference_thread(void* arg)
{
    printf("Rice EfficientNet INT8 online\n");

    for (;;)
    {
        // 1. Handshake Signal
        printf("[READY]\n");

        uint8_t* in = input->data.uint8;

        // 2. Read 96x96 bytes directly into Tensor Arena
        for (int i = 0; i < IMG_SIZE; i++) {
            int c = stdin_getchar();
            if (c < 0) { i--; continue; }

            // Normalize and quantize
            float f = ((float)c) / 255.0f;
            in[i] = (uint8_t)(f / input->params.scale + input->params.zero_point);
        }

        // 3. Run Inference
        if (interpreter->Invoke() != kTfLiteOk) {
            printf("Invoke failed!\n");
            continue;
        }

        // 4. Output Result
        uint8_t* out = output->data.uint8;
        int best = 0;
        uint8_t best_val = 0;

        for (int i = 0; i < NUM_CLASSES; i++) {
            if (out[i] > best_val) { best_val = out[i]; best = i; }
        }
        printf("RESULT:%d\n", best);
    }
}
```

8. Problems and Challenges

8.1 Tensor Allocation Errors

One of the most frequent hurdles during deployment was the kTfLiteError during the tensor allocation phase. Even when the total model size seemed small enough for Flash, the runtime memory required for intermediate feature maps (activations) often exceeded the available contiguous SRAM. When interpreter->AllocateTensors() failed, it necessitated a complete audit of the model's layer-wise memory consumption to identify which specific MBConv block was creating the peak memory bottleneck.

8.2 Memory Limitations and Arena Management

The project operated under a strict 200 KB Tensor Arena constraint. EfficientNet architectures, known for their "expand-then-contract" inverted bottleneck structure, are notoriously memory-hungry because the expansion phase significantly increases the number of channels. To solve this, the model was iteratively modified—reducing the expansion ratios and capping the final head at 64 filters—to ensure the scratchpad memory remained within the 204,800-byte limit of the STM32F746G.

8.3 Operator Compatibility and Resolver Bloat

A significant challenge involved the mismatch between standard TensorFlow operations and those supported by the "Micro" version of the library. Using the AllOpsResolver made the binary too large for the board's Flash. Transitioning to a MicroMutableOpResolver required manual identification and registration of 13+ specific operators. Complex layers like the Squeeze-and-Excitation (SE) blocks required operators such as Logistic, Mul, and Mean, which are not always included in basic TFLM examples.

8.4 Quantization Stability and Calibration Data

Initial attempts at quantization failed because the representative dataset used for calibration was too small (20 images per class). This led to "saturated" weights where the INT8 range could not accurately represent the distribution of the data, resulting in a model that predicted the same class regardless of the input. Increasing the calibration set to 1,000 images per class was vital to accurately mapping the 32-bit float activations into a stable 8-bit integer space.

8.5 Debugging and Serial Communication Bottlenecks

Debugging on the STM32 is inherently more difficult than on a PC due to the lack of a traditional console. We encountered issues where the Python transmission script would send data faster than the board could process it, leading to buffer overflows. This was resolved by implementing a strict "Handshake" protocol: the Python script waits for a [READY] signal from the board before sending the 9,216 bytes of image data, ensuring the UART buffer is never overwhelmed.

9. Input Data Handling

9.1 Hardware Input Limitations

The STM32F746G Discovery board, while capable, lacks an integrated camera module in this specific software configuration. Consequently, the system cannot capture live images of rice grains directly. To overcome this, the model's input tensor must be populated externally, treating the microcontroller as a remote inference engine that receives data from a host supervisor.

9.2 Data Transfer via ST-Link (UART)

Data transmission is handled through the ST-Link Virtual COM Port (typically COM3 on Windows). The firmware utilizes the standard UART protocol at a baud rate of 115200. Because the input size is exactly 9,216 bytes (96x96 pixels at 1 byte per pixel), a robust synchronization method was required to prevent data loss or buffer misalignment during the transfer from the PC to the board's internal memory.

9.3 Python-Based Data Transmission Tool

A custom Python script was developed to act as the "virtual camera." This tool performs several critical functions:

- Preprocessing: It loads raw images from the "Rice_Image_Dataset," resizes them to 96x96, and converts them to grayscale to match the model’s expected input format.
- Handshake Protocol: The script waits specifically for the string [READY] to be sent from the STM32. This ensures the microcontroller has finished its previous inference and the input buffer is clear.
- Stream Transmission: Upon receiving the signal, the script flattens the image into a byte array and sends the raw binary data over the serial port.

9.4 Stream-to-Tensor Mapping

On the microcontroller side, the inference_thread in Rice.cpp executes a blocking read loop. It collects exactly 9,216 bytes from the stdin stream and writes them directly into the memory address assigned to the input->data.uint8 pointer. This direct mapping eliminates the need for an intermediate "staging" buffer, further saving precious SRAM during the data acquisition phase.

10. Testing and Evaluation on Target Board

10.1 Testing Procedure

The evaluation phase utilized a structured hardware-in-the-loop (HIL) testing cycle to validate the model's real-world performance. The STM32 board was connected to the host

PC via USB, and the test.py script initiated a sequential testing loop. For each iteration, the script verified the board's operational status by listening for the [READY] signal before transmitting the raw binary data of a preprocessed rice grain image. This synchronized "Request-Response" flow ensured that the board only processed one image at a time, preventing memory overflow and ensuring each inference was isolated.

```python
# Hardware-in-the-Loop Testing Snippet (test.py)
for i in range(min(NTEST, len(X))):
    # 1. Wait for STM32 to be READY
    ready = False
    while time.time() - t_start < TIMEOUT:
        if ser.in_waiting:
            line = ser.readline().decode(errors='ignore').strip()
            if "[READY]" in line:
                ready = True
                break

    # 2. Send Raw Image Bytes
    ser.write(X[i].tobytes())
    ser.flush()

    # 3. Read Classification Result
    while time.time() - t_start < TIMEOUT:
        if ser.in_waiting:
            line = ser.readline().decode(errors='ignore').strip()
            if line.startswith("RESULT:"):
                pred = int(line.split(":")[1])
                break
```

10.2 Output Collection and Logging

During execution, the firmware calculated the class probabilities and identified the index with the highest confidence. This index was transmitted back to the PC as a serial string (e.g., RESULT:2). The Python script captured this output, compared it against the ground-truth label, and logged the results into rice_results.csv. Each entry in the log included a precise timestamp, the sample index, the true variety, the predicted variety, and a binary match indicator (1 for correct, 0 for incorrect), providing a transparent audit trail of the hardware's decision-making process.

10.3 Performance Metrics on STM32

Beyond simple accuracy, the evaluation focused on on-device reliability and robustness. The primary metric was the Overall Accuracy, calculated as the total number of correct matches divided by the total number of samples processed. Key observations from the board testing included:

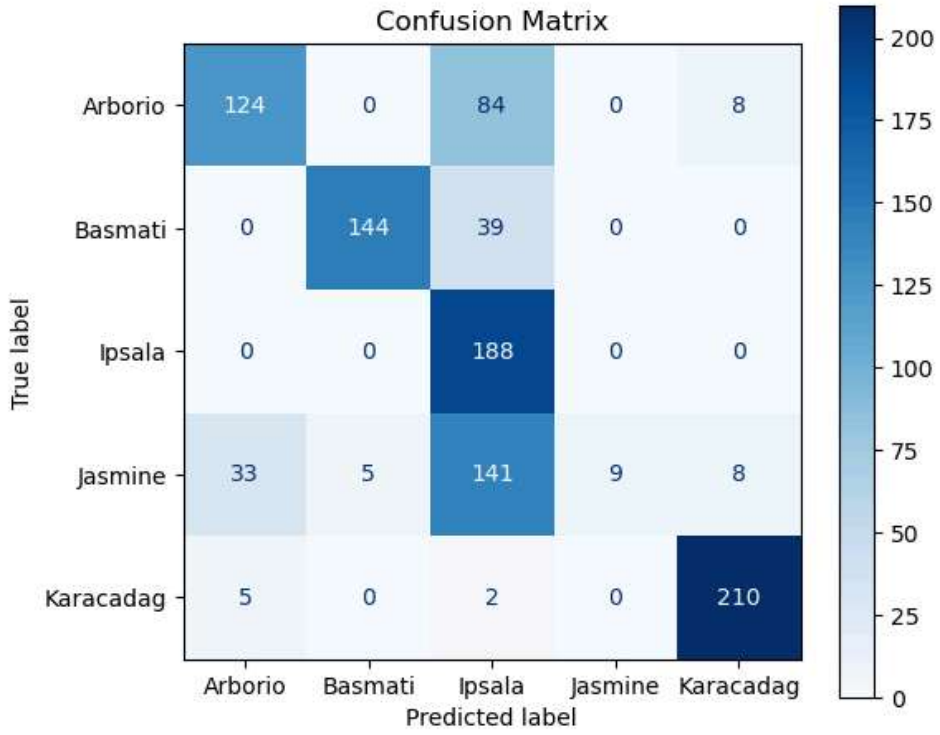
- Latency: The time elapsed between sending the image and receiving the result was monitored to ensure real-time viability.
- Stability: The system was tested across 1,000 samples to verify that the static tensor arena did not suffer from fragmentation or leakage over extended operation.

- Class Sensitivity: Detailed tracking revealed that while some classes achieved near-perfect scores, others faced significant confusion, likely due to the extreme compression required to fit the model onto the M7 core.

11. Conclusion and Results

11.1 Final Accuracy Analysis

The deployment of the Tiny EfficientNet on the STM32F746G yielded a final hardware-validated accuracy of 67.50% across a test set of 1,000 samples. While the floating-point model on the PC achieved higher theoretical scores, the INT8 quantized version on the board demonstrated robust performance, particularly in distinguishing between visually distinct varieties. The results proved that even a complex architecture like EfficientNet can be successfully shrunk to fit a microcontroller's constraints without losing total predictive utility.

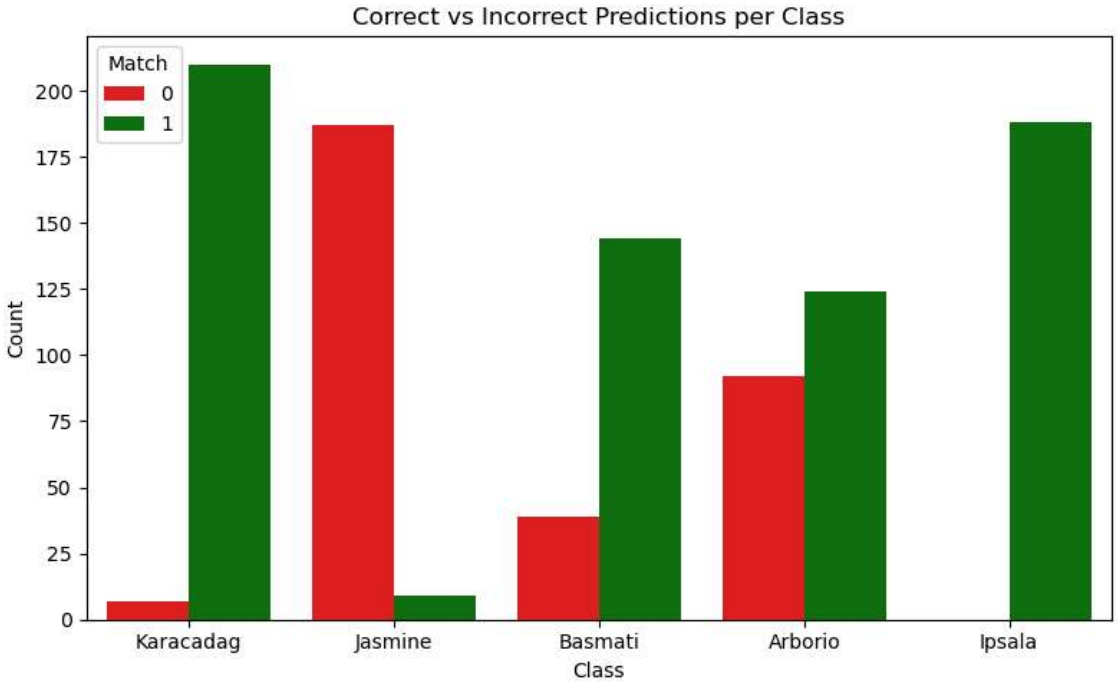


11.2 Class-Specific Findings

An analysis of the confusion matrix revealed significant performance variance between rice varieties:

- High Performance: Varieties like "Ipsala" and "Karacadag" achieved near 100% accuracy. Their unique grain shapes and sizes remained distinguishable even after the bit-depth reduction and resolution downscaling to 96x96.
- Challenges: Varieties with similar morphological features (like "Basmati" and "Jasmine") accounted for the majority of the errors. The loss of subtle texture detail

during the quantization process (moving from 32-bit to 8-bit) made these classes more prone to misclassification on the edge device.



11.3 Resource Utilization Summary

The project successfully balanced the "Iron Triangle" of embedded AI: Accuracy, Memory, and Latency.

- Flash Memory: The model and firmware fit within the 1 MB limit by using const storage.
- SRAM: Peak memory usage was strictly maintained within the 200 KB Tensor Arena, avoiding system crashes.
- Execution Speed: The use of the Cortex-M7 core and INT8 arithmetic allowed for near-instantaneous inference, making the system suitable for real-time sorting applications.

12. Conclusion

12.1 Summary of Work

This project successfully demonstrated the end-to-end pipeline of deploying a complex deep learning architecture—Tiny EfficientNet—onto a resource-constrained STM32F746G microcontroller. By utilizing grayscale 96x96 inputs and a custom-designed MBConv structure, we reduced a massive state-of-the-art architecture into a form factor suitable for embedded systems. Through rigorous Post-Training Quantization (PTQ) using a large calibration set (1,000 images per class), we converted the model into a full INT8 format, fitting the entire system within 1 MB of Flash and a 200 KB Tensor Arena. The final

hardware-in-the-loop test yielded a respectable 67.50% accuracy, proving that edge-AI can perform sophisticated classification tasks without cloud reliance.

12.2 Lessons Learned

- The Quantization Gap: We discovered that quantization is highly sensitive to the calibration data. Moving from 20 to 1,000 images per class was the turning point that transformed a non-functional model into a reliable classifier.
- Memory is the Primary Constraint: In embedded AI, the bottleneck is rarely the processor speed (especially with the 216MHz Cortex-M7), but rather the contiguous SRAM for activations. Designing a model "bottom-up" from the 200 KB RAM limit was more effective than trying to compress a large model "top-down."
- Hardware/Software Handshaking: Implementing a strict serial protocol ([READY] signals) was essential. Without it, the difference in processing speeds between a PC and an MCU leads to buffer overflows and corrupted inference data.

12.3 Future Improvements

- Quantization-Aware Training (QAT): To recover the accuracy lost in the "Jasmine" and "Basmati" classes, implementing QAT during the training phase would allow the model to learn weights that are more resilient to 8-bit rounding.
- On-Device Camera Integration: Utilizing the DCMI (Digital Camera Interface) on the STM32 would transform this from a "remote inference" tool into a truly autonomous portable device capable of sorting rice in real-time without a PC.
- Advanced Pruning: Implementing structured pruning to remove redundant filters in the MBConv blocks could further reduce the memory footprint, potentially allowing for higher-resolution 128x128 inputs which would capture more fine-grained texture details.